ANTEPROYECTO

Sistema web de gestión de incidencias informáticas para pequeñas organizaciones

Proyecto Intermodular de Administración de Sistemas Informáticos en Red

Ciclo Formativo de Grado Superior – Técnico Superior en Administración de Sistemas Informáticos en Red

Curso 2026/2027

Versión actualizada – Entregable 3: planificación y organización del proyecto

Alumna: Elida Martinez López

Departamento de Informática

Índice

Introducción

1. Título

2. Descripción

3. Justificación

4. Objetivos

5. Lenguajes de programación y tecnologías

6. Prototipo de interfaz de usuario

7. Gestor de bases de datos

8. Entornos de aplicación del proyecto

9. Fases de desarrollo

10. Módulos del ciclo formativo relacionados

11. Anexos y bibliografía

- Anexo I. Esquema de red del entorno de pruebas
- Anexo II. Plan de direccionamiento y servicios
- Anexo III. Matriz de permisos por rol
- Anexo IV. Esquema del script de copias de seguridad
- Anexo V. Enlaces del proyecto
- Bibliografía

Introducción

Este documento recoge el anteproyecto del Proyecto Intermodular del ciclo de Administración de Sistemas Informáticos en Red. En él se presenta la idea del proyecto, el problema que pretende resolver, sus objetivos y las decisiones técnicas tomadas para llevarlo a cabo: tecnologías, estructura de la base de datos, primeras pantallas de la aplicación, infraestructura necesaria y planificación del trabajo.

El proyecto combina dos partes que se trabajan durante el ciclo: por un lado, el desarrollo de una aplicación web y, por otro, su despliegue y administración sobre una infraestructura propia (servidor, red, seguridad, copias de seguridad y monitorización). El anteproyecto sirve como base para que el profesor valide el planteamiento antes de iniciar la fase de ejecución.

1. Título

Sistema web de gestión de incidencias informáticas para pequeñas organizaciones

2. Descripción

El proyecto consiste en el desarrollo de un sistema web para la gestión de incidencias informáticas dirigido a pequeñas organizaciones. Su objetivo principal será centralizar el registro y seguimiento de los problemas técnicos reportados por los usuarios, evitando que estas incidencias se gestionen de forma dispersa mediante mensajes, llamadas, correos electrónicos o comunicaciones verbales.

El sistema permitirá registrar incidencias relacionadas con hardware, software, red, cuentas de usuario u otros recursos informáticos y consultar su estado. El personal técnico podrá clasificarlas mediante categorías fijas, establecer su prioridad, actualizar su estado, añadir comentarios de seguimiento, adjuntar o consultar evidencias y registrar la solución aplicada. Las incidencias podrán requerir tanto soporte remoto como intervención presencial. De esta forma, se pretende facilitar la organización del soporte informático, mantener un historial de las incidencias y mejorar su seguimiento hasta la resolución.

3. Justificación

En pequeñas organizaciones es habitual que los problemas informáticos se comuniquen directamente al responsable de soporte mediante distintos canales. Esta forma de trabajo puede provocar pérdida de información, falta de seguimiento, desconocimiento del estado de una incidencia o dificultad para consultar posteriormente cómo se resolvió un problema similar.

El proyecto pretende responder a esta necesidad mediante un sistema centralizado en el que cada incidencia quede registrada desde su creación hasta su resolución, conservando información sobre el usuario que la reportó, su categoría, prioridad, estado, comentarios, evidencias y solución aplicada.

Existen soluciones profesionales para esta finalidad, como osTicket, GLPI, Zammad o Jira Service Management. Por tanto, el proyecto no se plantea porque no existan herramientas capaces de resolver el problema. En una empresa real sería necesario valorar primero la implantación de una solución existente.

La propuesta de desarrollar una solución propia se justifica principalmente por su finalidad formativa y por el control del alcance. Permitirá diseñar directamente el modelo de datos, los

usuarios y permisos, el flujo de las incidencias y las funcionalidades de la aplicación y, posteriormente, desplegarla y administrarla sobre una infraestructura propia. La aplicación no pretende competir con soluciones profesionales, sino implementar un conjunto reducido de funciones que permita trabajar de forma integrada contenidos de desarrollo web, bases de datos, sistemas operativos, servidores, redes, seguridad, copias de seguridad y monitorización.

Estado del arte

Antes de plantear el proyecto se han revisado las principales herramientas de gestión de incidencias que existen en el mercado. La siguiente tabla resume sus características y en qué se diferencia la propuesta de este proyecto:

Comparativa de soluciones existentes

Solución	Tipo y licencia	Puntos fuertes	Limitaciones para una organización pequeña
osTicket	Código abierto (GPL), autoalojado. PHP + MySQL.	Sistema de tickets veterano, recepción por correo y web, muy extendido.	Interfaz anticuada y muchas opciones que no se necesitan en un entorno pequeño.
GLPI	Código abierto (GPL), autoalojado. PHP + MySQL/MariaDB.	Combina incidencias con inventario de equipos (ITSM completo).	Curva de aprendizaje alta; pensado para departamentos de IT medianos o grandes.
Zammad	Código abierto (AGPL) o versión en la nube de pago. Ruby on Rails.	Interfaz moderna y multicanal (correo, chat, redes sociales).	Requisitos de servidor elevados (Elasticsearch, Redis) para un uso sencillo.
Jira Service Management	Comercial (Atlassian), en la nube; plan gratuito limitado.	Muy completo, con SLA, automatizaciones e integraciones.	Coste por agente al crecer, dependencia de un proveedor externo y datos fuera de la organización.
Este proyecto	Desarrollo propio, autoalojado. PHP (Laravel) + MySQL.	Solo las funciones necesarias, interfaz sencilla y control total del despliegue y de los datos.	Menos funciones que las soluciones profesionales; su objetivo es formativo y de alcance controlado.

La principal diferencia de la propuesta es que el proyecto abarca todo el ciclo de vida del servicio: no solo la aplicación, sino también el servidor donde se aloja, su seguridad, las copias de seguridad y la monitorización. Esto permite demostrar las competencias propias de un administrador de sistemas, que es el perfil del ciclo.

4. Objetivos

Objetivo general

Diseñar, desarrollar e implementar un sistema web que permita registrar, gestionar y realizar el seguimiento de incidencias informáticas dentro de pequeñas organizaciones.

Objetivos específicos

- Implementar un sistema de autenticación para identificar a los usuarios.

- Definir roles y permisos diferenciados para usuarios y personal técnico o administrador.
- Permitir a los usuarios registrar nuevas incidencias informáticas.
- Clasificar las incidencias mediante categorías fijas y asignarles prioridad y estado.
- Permitir al personal técnico consultar, gestionar y actualizar las incidencias recibidas.
- Incorporar comentarios de seguimiento dentro de cada incidencia.
- Permitir adjuntar archivos como evidencias relacionadas con una incidencia.
- Registrar la solución aplicada y conservar un historial de las incidencias gestionadas.
- Diseñar e implementar una base de datos para almacenar la información del sistema.
- Desplegar la aplicación sobre un servidor web.
- Aplicar medidas básicas de seguridad para proteger el acceso al sistema y la información almacenada.
- Implementar copias de seguridad de los datos y archivos necesarios.
- Incorporar mecanismos básicos de monitorización del servidor y de los servicios utilizados.

Objetivos concretos de infraestructura

Para que los objetivos anteriores sean medibles, se concretan en las siguientes tareas de administración de sistemas:

- Instalar y configurar un servidor Ubuntu Server 26.04 LTS con Apache 2.4, PHP 8.5 y MySQL 8.4 LTS para alojar la aplicación.
- Configurar el acceso a la aplicación únicamente por HTTPS, con redirección automática desde HTTP.
- Configurar el cortafuegos UFW dejando abiertos solo los puertos necesarios y proteger SSH con autenticación por clave y Fail2ban.
- Programar con cron una copia de seguridad diaria de la base de datos y de los archivos adjuntos, enviarla a un segundo servidor y comprobar que se puede restaurar.
- Instalar Zabbix para vigilar el estado del servidor (CPU, memoria, disco) y de los servicios Apache y MySQL, con alertas cuando algo falle.

5. Lenguajes de programación y tecnologías

Para el desarrollo y la puesta en marcha del proyecto se ha definido el siguiente conjunto de tecnologías. Se han agrupado en dos bloques: las que se usan para desarrollar la aplicación y las que forman la infraestructura donde se despliega y administra.

Desarrollo de la aplicación

PHP 8.5: Será el lenguaje principal del backend. Se ha elegido porque forma parte de los contenidos que la alumna trabajará durante el curso y permite reforzar los conocimientos adquiridos mediante un proyecto real. Se usará la versión 8.5, que es la última versión estable y la que incluye Ubuntu Server 26.04 LTS. La versión 8.6 está prevista para finales de 2026; si se publica durante el desarrollo y Laravel la admite, se valorará actualizar.

Laravel 13: Se utilizará como framework de PHP, en su versión 13 (la más reciente, publicada en 2026, que requiere PHP 8.3 o superior). Proporciona una estructura organizada para el desarrollo y herramientas para tareas necesarias en el proyecto, como autenticación, autorización, acceso a bases de datos, validación, migraciones y gestión de archivos.

MySQL 8.4 LTS: Será el sistema gestor de bases de datos relacional. Se usará la versión 8.4, que es la versión de soporte a largo plazo actual de MySQL. Almacenará usuarios,

incidencias, comentarios (con la ruta de los archivos adjuntos), el historial de estados y demás información del sistema.

HTML: Se utilizará para definir la estructura de las páginas y formularios de la aplicación web.

CSS: Se utilizará para personalizar la presentación visual. Parte del estilo se desarrollará manualmente para adaptar la interfaz a las necesidades del proyecto.

JavaScript: Se utilizará para incorporar interactividad en aquellas partes de la interfaz que lo requieran.

Blade: Se utilizará como sistema de plantillas de Laravel para construir las vistas de la aplicación y reutilizar elementos comunes de la interfaz.

Bootstrap 5.3: Será la base principal para el diseño responsive y para componentes como formularios, botones, tablas, alertas, tarjetas, badges y modales. Permitirá reducir tiempo de maquetación sin impedir el uso de CSS propio.

AdminLTE 4: Se utilizará de forma parcial, principalmente en vistas de administración o del personal técnico, aprovechando componentes y estructuras orientadas a dashboards sin depender por completo de una plantilla prefabricada.

Composer, Node.js y Vite: Composer gestionará las dependencias de PHP y Laravel; Node.js y Vite (incluido en Laravel) se usarán para compilar y optimizar los archivos CSS y JavaScript antes del despliegue.

Git y GitHub: Se usará Git para el control de versiones del código y un repositorio privado en GitHub como copia remota. Además, facilitará el despliegue: el servidor descargará la versión final de la aplicación desde el repositorio.

Infraestructura, sistemas y servicios

Ubuntu Server 26.04 LTS: Sistema operativo del servidor. Es la versión con soporte a largo plazo más reciente (publicada en abril de 2026, con actualizaciones de seguridad hasta 2031), muy documentada y con la que se ha trabajado en distintas prácticas del ciclo, lo que la hace adecuada para un servidor estable.

Apache HTTP Server 2.4 con PHP-FPM: Servidor web que publicará la aplicación. Se configurará un VirtualHost que apunte a la carpeta pública de Laravel, con los módulos mod_rewrite (rutas amigables) y mod_ssl (HTTPS). PHP se ejecutará mediante PHP-FPM, que separa el procesamiento de PHP del servidor web y mejora el rendimiento. Se ha elegido Apache frente a Nginx porque es el servidor trabajado en el módulo de Servicios de Red e Internet y Laravel funciona perfectamente con él.

VirtualBox: Plataforma de virtualización para montar el entorno de pruebas completo (servidores y clientes) en el equipo de la alumna. Se usará una Red NAT para que las máquinas se vean entre sí y tengan salida a Internet, e instantáneas (snapshots) antes de cada cambio importante para poder volver atrás si algo falla.

OpenSSL (HTTPS): Como el entorno es local y no tiene un dominio público, se creará una autoridad de certificación propia con OpenSSL para emitir el certificado del servidor e importarla en los clientes, de forma que el navegador confíe en la conexión. En un despliegue real con dominio público se utilizaría Let's Encrypt.

UFW (cortafuegos): Se configurará con política de denegar todo el tráfico entrante por defecto y permitir solo lo necesario: HTTP/HTTPS para la aplicación, SSH únicamente desde la red interna y el puerto del agente de monitorización solo desde el servidor de monitorización. MySQL no será accesible desde fuera del propio servidor.

OpenSSH y Fail2ban: La administración remota se hará por SSH con autenticación mediante clave y sin permitir el acceso directo como root. Fail2ban bloqueará temporalmente las IP que acumulen intentos fallidos de inicio de sesión.

Bash, cron y rsync (copias de seguridad): Un script en Bash exportará la base de datos con mysqldump y empaquetará los archivos adjuntos y la configuración de la aplicación. Se ejecutará cada día con cron y enviará la copia al segundo servidor mediante rsync sobre SSH, eliminando las copias más antiguas (rotación).

Zabbix 7.0 LTS (monitorización): Herramienta de monitorización de código abierto. Se usa la versión 7.0 por ser la LTS vigente; la 8.0 LTS está anunciada para finales de 2026 y se valorará usarla si ya está publicada al montar el servidor. El servidor Zabbix se instalará en una segunda máquina y el agente en el servidor web, para vigilar CPU, memoria, disco, red y el estado de Apache y MySQL, mostrando gráficas y generando alertas.

Decisiones funcionales del sistema

- El sistema gestionará incidencias informáticas relacionadas, entre otras, con hardware, software, red, cuentas de usuario y periféricos.
- Una incidencia podrá requerir atención remota o intervención presencial; el sistema gestiona el seguimiento del caso, no limita la forma en que el técnico realiza la reparación o asistencia.
- Existirán dos perfiles fijos: usuario y técnico/administrador. Como no se van a crear ni modificar desde la aplicación, se guardarán en un campo del propio usuario.
- Las categorías de incidencias serán fijas en la primera versión del proyecto: Hardware, Software, Red, Cuentas de usuario, Periféricos y Otros.
- Como los perfiles, las categorías, las prioridades y los estados son valores fijos, no tendrán tablas propias en la base de datos: se guardarán como campos de lista de valores (ENUM) en las tablas USUARIOS e INCIDENCIAS.
- Las prioridades serán Baja, Media, Alta y Crítica. El usuario describe el problema y es el personal técnico quien asigna la prioridad definitiva.
- El flujo de estados será: Abierta → Asignada → En curso → (En espera) → Resuelta → Cerrada. El usuario podrá confirmar la solución o reabrir la incidencia si el problema continúa.
- El estado **En espera** se usará cuando la incidencia no puede avanzar porque falta algo necesario. Por ejemplo, si para seguir hay que comprar un cargador nuevo para un portátil, la incidencia queda en espera hasta que llega el cargador y después vuelve a En curso.
- Cada incidencia podrá incluir comentarios de seguimiento; el técnico podrá marcar comentarios como internos, visibles solo para el personal técnico.
- Se podrán adjuntar archivos como evidencias relacionadas con una incidencia, con tipos y tamaño máximo limitados (imágenes y PDF, hasta 5 MB). Cada archivo irá asociado a un comentario de la incidencia.
- El sistema mantendrá un historial del proceso desde el registro de la incidencia hasta su resolución y cierre, guardando cada cambio de estado, quién lo hizo y cuándo.

6. Prototipo de interfaz de usuario

A continuación se muestra un prototipo inicial de las pantallas principales de la aplicación. Son maquetas de diseño: sirven para visualizar la experiencia del usuario y del técnico antes de programar, por lo que los colores, textos y la distribución podrán ajustarse durante el desarrollo. Todos los nombres y datos que aparecen son ficticios.

La interfaz sigue el estilo de un panel de administración (basado en Bootstrap y AdminLTE): un menú lateral con las opciones de cada perfil, una barra superior con el usuario conectado y el contenido principal en tarjetas. Al usar Bootstrap, las pantallas se adaptarán también a móvil y tableta.

Se incluye una maqueta por cada opción del menú. El usuario tiene las opciones Mis incidencias, Nueva incidencia, Mi perfil y Cerrar sesión; el técnico tiene Panel, Todas las incidencias, Asignadas a mí, Usuarios, Informes y Cerrar sesión. Todas las maquetas están también en el repositorio de GitHub (docs/diseno/prototipos), junto con sus archivos HTML editables.

Pantalla de inicio de sesión

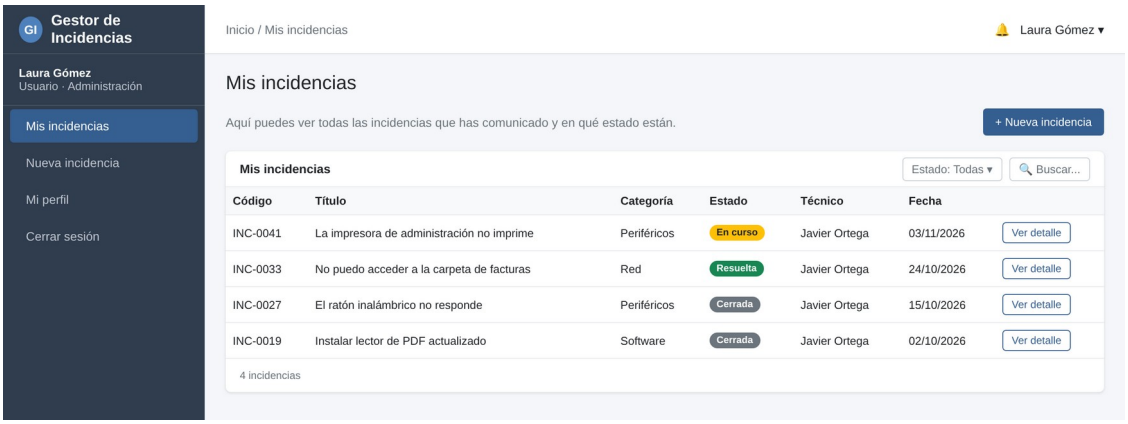


Pantalla de inicio de sesión

Es la puerta de entrada al sistema. El usuario se identifica con su correo y contraseña y la aplicación le lleva a su panel de usuario o al panel técnico, según su perfil. El acceso se hace siempre por HTTPS.

Pantallas del usuario

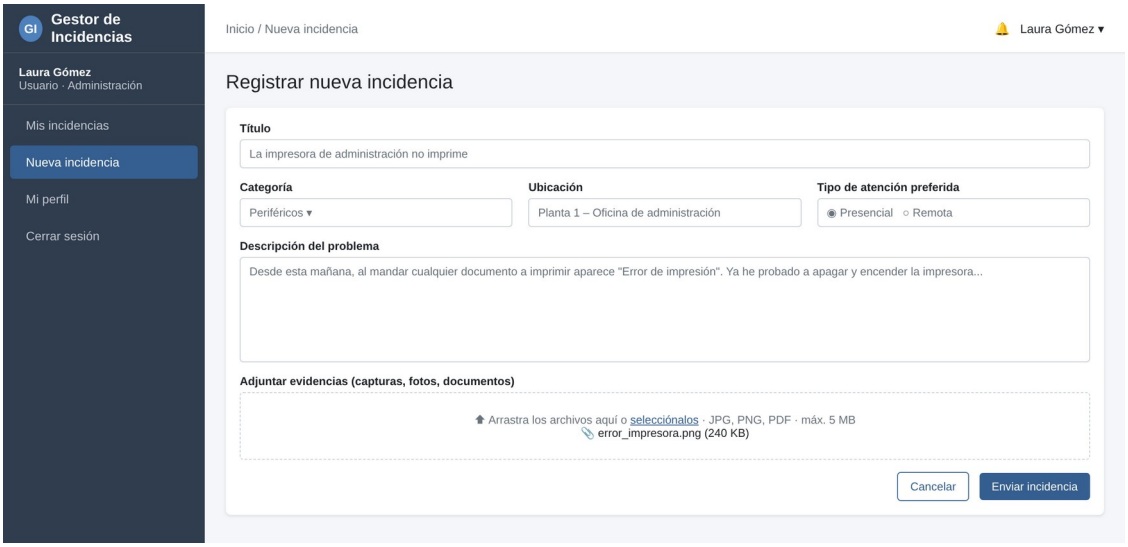
Mis incidencias



Listado de las incidencias del usuario

Es lo primero que ve el empleado al entrar. Aparecen todas las incidencias que ha comunicado, con su estado y el técnico que la lleva, para que sepa en qué punto está cada una sin tener que preguntar. Desde aquí puede abrir el detalle de cada incidencia o crear una nueva.

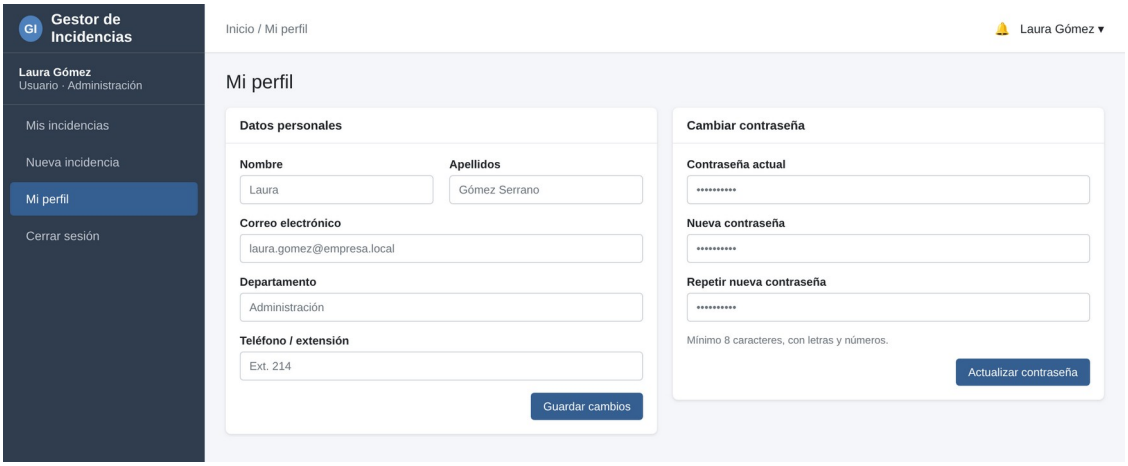
Nueva incidencia



Formulario para registrar una nueva incidencia

Desde aquí cualquier empleado puede comunicar un problema: indica un título, la categoría, dónde está, si prefiere atención presencial o remota y una descripción. También puede adjuntar una captura o una foto del error, lo que ayuda mucho al técnico a entender el problema sin tener que desplazarse.

Mi perfil

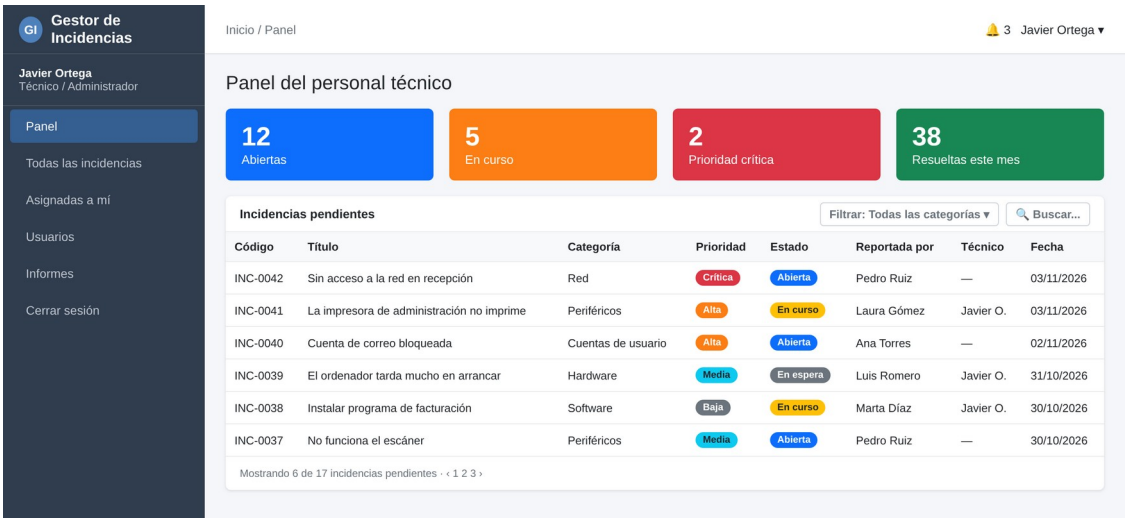


Datos personales y cambio de contraseña

Aquí el usuario puede revisar y corregir sus datos de contacto y cambiar su contraseña. Es importante que los datos estén al día, porque el técnico los usa para localizarle cuando tiene que resolver una incidencia.

Pantallas del técnico

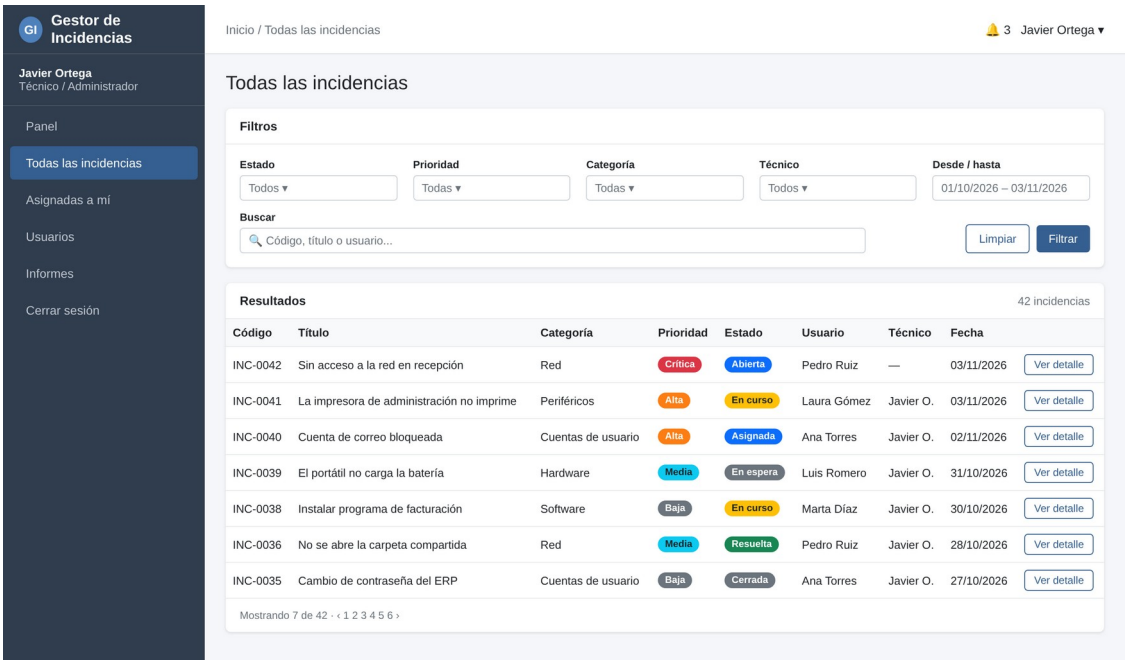
Panel



Panel principal del técnico con resumen e incidencias pendientes

Es la pantalla principal del técnico. Arriba se ven de un vistazo los indicadores más importantes (incidencias abiertas, en curso, críticas y resueltas este mes) y debajo la lista de incidencias pendientes, con colores según la prioridad y el estado, y opciones para filtrar y buscar.

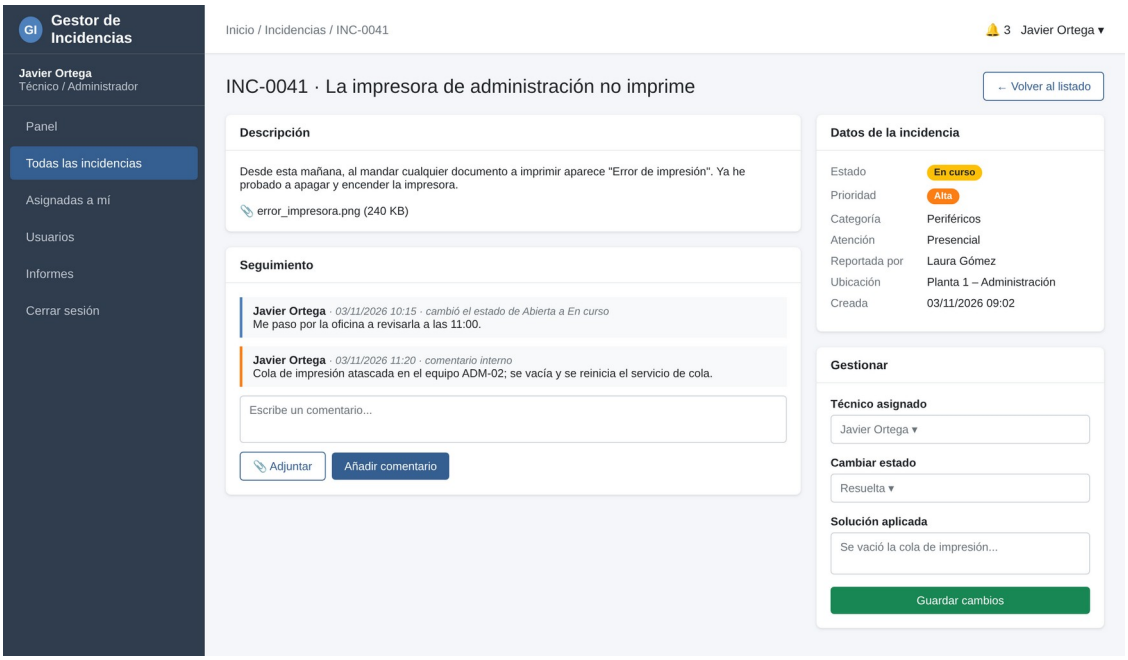
Todas las incidencias



Listado de todas las incidencias con filtros

Muestra un listado con todas las incidencias de la organización y un bloque de filtros para buscar por estado, prioridad, categoría, técnico asignado y fechas, o por texto. Cada fila tiene el botón «Ver detalle», que abre la ficha completa de esa incidencia.

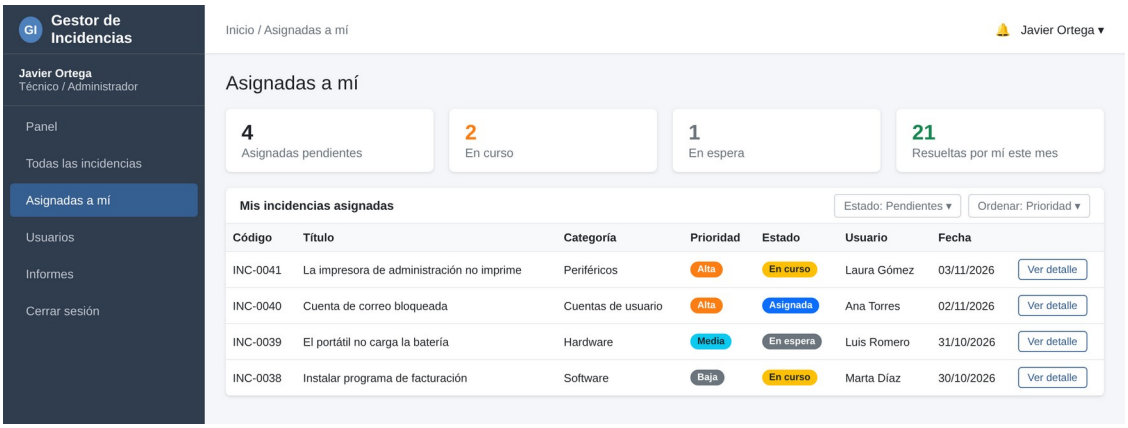
Detalle de una incidencia



Detalle de una incidencia con su seguimiento y opciones de gestión

Se abre al pulsar «Ver detalle» en cualquier listado, y desde ella se puede volver con el botón «Volver al listado». Aquí se trabaja cada incidencia. A la izquierda aparece la descripción, los archivos adjuntos y el historial de comentarios; a la derecha, los datos de la incidencia y el bloque para asignar técnico, cambiar el estado y escribir la solución aplicada.

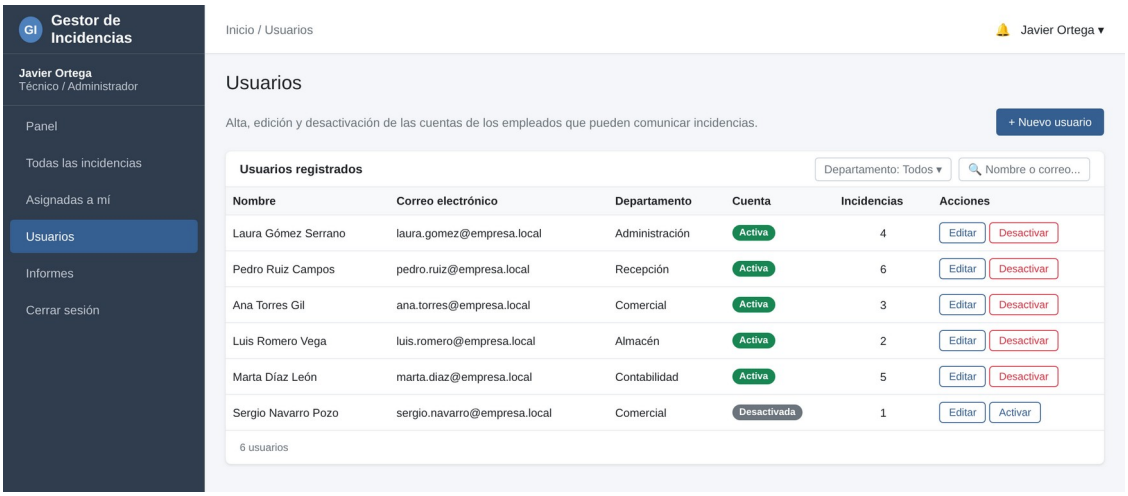
Asignadas a mí



Incidencias asignadas al técnico conectado

Es la lista de trabajo del técnico: solo aparecen las incidencias que tiene asignadas, ordenadas por prioridad, con un pequeño resumen de cuántas tiene en curso, cuántas en espera y cuántas ha resuelto este mes. Así sabe en todo momento qué tiene pendiente y por dónde empezar.

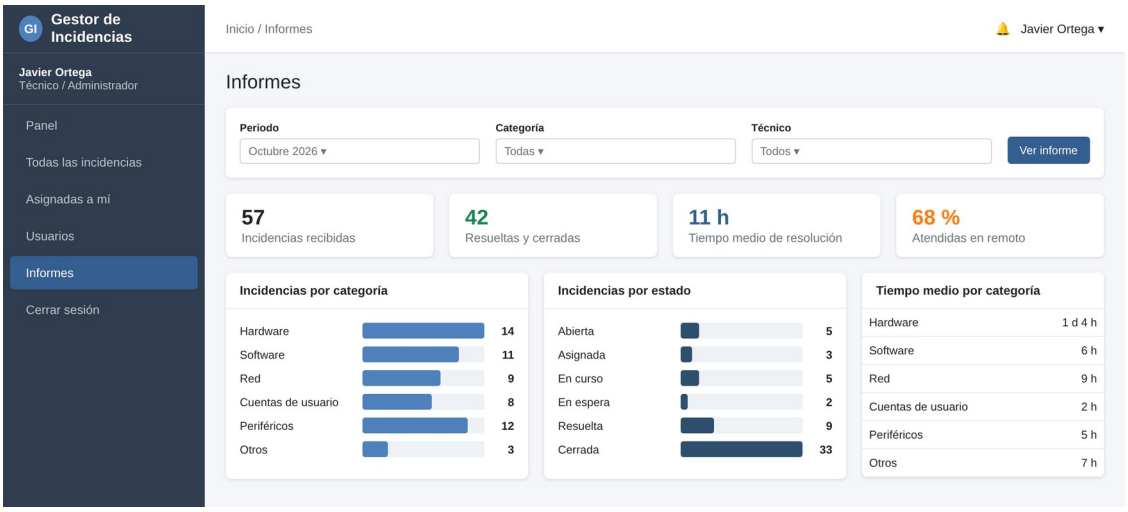
Usuarios



Gestión de las cuentas de usuario

Desde aquí el técnico da de alta las cuentas de los empleados, corrige sus datos y desactiva las de quienes ya no trabajan en la organización. Las cuentas no se borran, solo se desactivan, para no perder el historial de las incidencias que comunicaron.

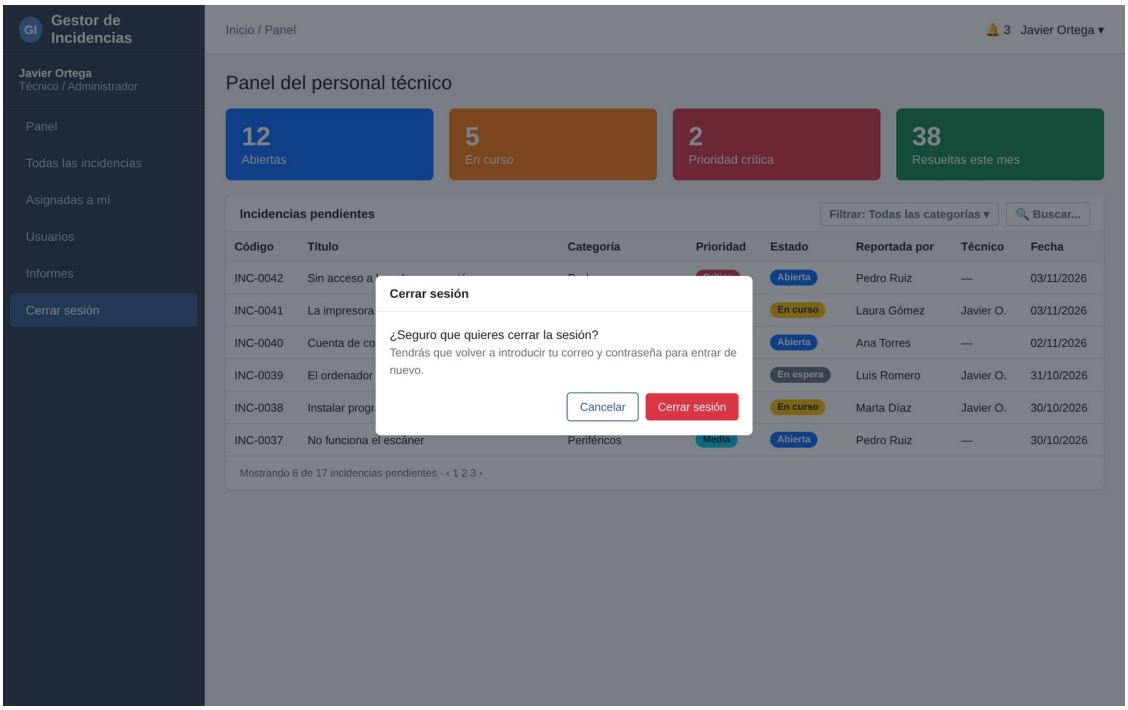
Informes



Informe de incidencias por periodo

Muestra un resumen del soporte en el periodo elegido: cuántas incidencias han llegado, cuántas se han resuelto, el tiempo medio de resolución y qué parte se ha atendido en remoto. Las gráficas por categoría y por estado ayudan a ver dónde se concentran los problemas, por ejemplo si la mayoría son de hardware o de red.

Cerrar sesión



Confirmación para cerrar la sesión

Al pulsar «Cerrar sesión» (en el menú del usuario o del técnico) aparece una ventana que pide confirmación, para evitar salir por error. Si se confirma, la sesión se cierra y la aplicación vuelve a la pantalla de inicio de sesión.

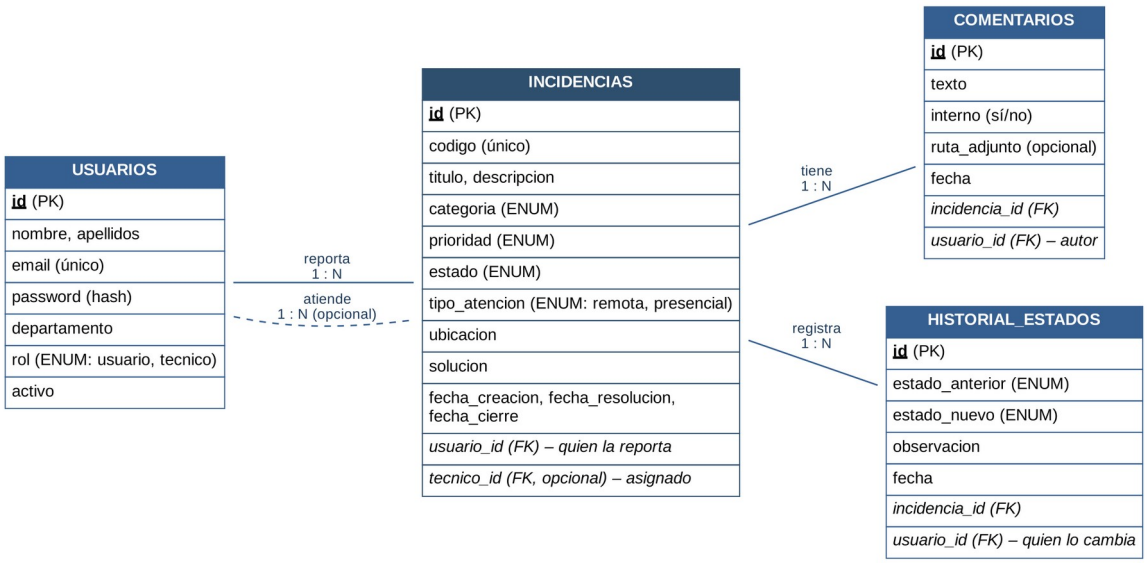
7. Gestor de bases de datos

El sistema gestor de bases de datos elegido es **MySQL 8.4 LTS**, instalado en el mismo servidor que la aplicación. Se ha elegido por los siguientes motivos:

- Es un SGBD relacional muy extendido, gratuito y con amplia documentación, y es el que se trabaja en los módulos de bases de datos del ciclo.
- El motor InnoDB permite usar claves foráneas e integridad referencial, necesarias para relacionar usuarios, incidencias, comentarios e historial.
- Laravel lo soporta de forma nativa: la estructura se creará con migraciones (versionadas en Git) y los datos iniciales con seeders.
- Se usará el juego de caracteres utf8mb4 para guardar correctamente tildes, eñes y cualquier otro carácter.

Por seguridad, MySQL solo escuchará en la propia máquina (127.0.0.1) y la aplicación se conectará con un usuario específico que solo tendrá permisos sobre su base de datos, nunca con el usuario root.

Modelo Entidad-Relación



Modelo Entidad-Relación de la base de datos

Descripción de las entidades

Entidad	Qué guarda
USUARIOS	Personas que acceden al sistema: datos personales, correo (único, sirve para iniciar sesión), contraseña cifrada, departamento, si la cuenta está activa y su rol. El rol es un campo de valores fijos (usuario o técnico).
INCIDENCIAS	Entidad principal. Cada incidencia tiene un código único (por ejemplo INC-0041), título, descripción, tipo de atención, ubicación, solución aplicada y fechas. La categoría, la prioridad y el estado se guardan como campos de valores fijos. Se relaciona con el usuario que la reporta y con el técnico asignado.
COMENTARIOS	Mensajes de seguimiento escritos dentro de una incidencia por el usuario o el técnico. El campo interno indica si es visible solo para el personal técnico. Si el comentario lleva un archivo adjunto, se guarda su ruta en la columna ruta_adjunto; el archivo se almacena en la zona privada de Laravel, no accesible directamente desde la web. Los archivos que se adjuntan al registrar la incidencia se guardan como su primer comentario.
HISTORIAL_ESTADOS	Registro de cada cambio de estado: estado anterior, estado nuevo, quién lo hizo, cuándo y una observación. Permite reconstruir todo el recorrido de una incidencia.

Relaciones

- Un **usuario** puede reportar muchas incidencias y cada incidencia la reporta un único usuario (1:N).
- Un **técnico** (usuario con rol técnico) puede tener asignadas muchas incidencias; una incidencia puede no tener técnico todavía o tener uno (1:N opcional).
- Una **incidencia** tiene muchos comentarios y muchos registros de historial (1:N). Cada comentario y cada registro de historial guarda también el usuario que lo creó.

Valores fijos: rol (usuario, técnico); categoría (Hardware, Software, Red, Cuentas de usuario, Periféricos, Otros); prioridad (Baja, Media, Alta, Crítica); estado (Abierta, Asignada, En curso, En espera, Resuelta, Cerrada). Si en el futuro hubiera que gestionarlos desde la aplicación, se pasarían a tablas propias.

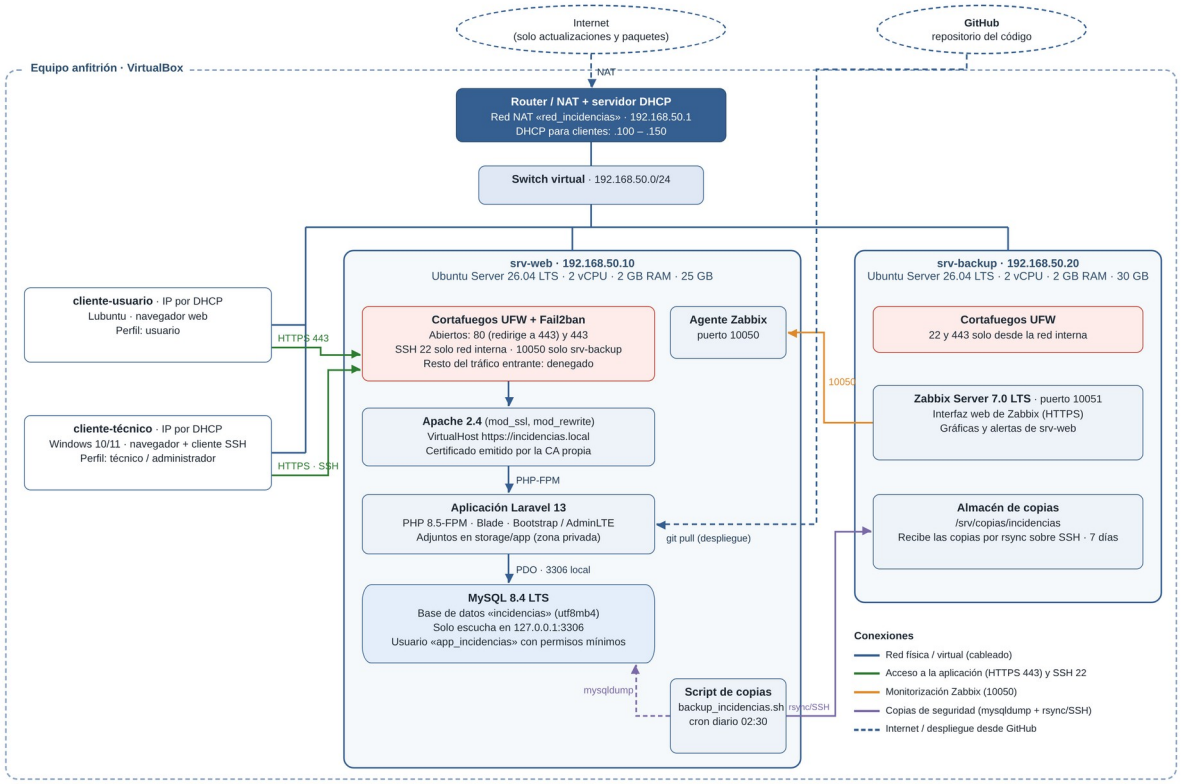
Además de estas tablas, Laravel crea sus propias tablas internas (sesiones, migraciones, recuperación de contraseñas, caché, etc.), que no se incluyen en el diagrama porque no forman parte del modelo del problema.

8. Entornos de aplicación del proyecto

El proyecto se trabajará en tres entornos: el de desarrollo (donde se programa), el de pruebas y despliegue (las máquinas virtuales que simulan la red de una pequeña organización) y la red que las conecta.

Arquitectura de la solución

El siguiente diagrama muestra todos los componentes del proyecto y cómo se comunican entre sí: los servidores y sus sistemas operativos, los servicios que corre cada uno, la red con sus direcciones IP, el router y el switch virtuales, los cortafuegos, la base de datos y los clientes que usan la aplicación.



Arquitectura e infraestructura de la solución

El funcionamiento es sencillo: los clientes se conectan por la red virtual a srv-web usando HTTPS. Antes de llegar a la aplicación, el tráfico pasa por el cortafuegos UFW, que solo deja pasar lo necesario. Apache recibe la petición y la entrega a Laravel a través de PHP-FPM, y Laravel consulta la base de datos MySQL, que está en el mismo servidor y no se puede alcanzar desde fuera. Por otro lado, srv-backup vigila a srv-web con Zabbix y guarda cada noche las copias de seguridad que le envía el script por rsync. El código de la aplicación se despliega descargándolo desde GitHub.

Especificación técnica de los componentes

Componente	Función	Sistema operativo o tecnología	Configuración prevista
Router / NAT + DHCP	Dar salida a Internet y repartir IP a los clientes	Red NAT de VirtualBox	Red 192.168.50.0/24 · puerta de enlace 192.168.50.1 · DHCP .100–.150
Switch virtual	Conectar todas las máquinas de la red	Red NAT de VirtualBox	Un único segmento 192.168.50.0/24
Servidor web (srv-web)	Alojar la aplicación	Ubuntu Server 26.04 LTS · Apache 2.4 · PHP 8.5-FPM · Laravel 13	IP fija 192.168.50.10 · 2 vCPU, 2 GB RAM, 25 GB · HTTPS en incidencias.local
Base de datos	Almacenar usuarios, incidencias, comentarios e historial	MySQL 8.4 LTS (en srv-web)	BD «incidencias» (utf8mb4) · escucha solo en 127.0.0.1:3306 · usuario «app_incidencias» con permisos mínimos
Cortafuegos de srv-web	Filtrar el tráfico de entrada y frenar ataques por fuerza bruta	UFW + Fail2ban	Permite 80/443; 22 solo desde la red interna; 10050 solo desde srv-backup; resto denegado
Servidor de administración (srv-backup)	Monitorizar y guardar las copias de seguridad	Ubuntu Server 26.04 LTS · Zabbix 7.0 LTS · rsync	IP fija 192.168.50.20 · 2 vCPU, 2 GB RAM, 30 GB · copias en /srv/copias/incidencias (7 días)
Cortafuegos de srv-backup	Proteger el servidor de administración	UFW	Permite 22 y 443 solo desde la red interna
Cliente usuario	Registrar y consultar incidencias	Lubuntu · navegador web	IP por DHCP · acceso por https://incidencias.local
Cliente técnico	Gestionar incidencias y administrar los servidores	Windows 10/11 · navegador web · cliente SSH	IP por DHCP · acceso por HTTPS y por SSH con clave
Repositorio GitHub	Guardar el código y la documentación; publicar la web	Git · GitHub · GitHub Pages	Repositorio público gestor-incidencias

Entorno de desarrollo

Software y hardware del entorno de desarrollo

Elemento	Uso
Equipo anfitrión con Windows	Ordenador de la alumna donde se programa y se ejecutan las máquinas virtuales. Recomendado: procesador de 4 núcleos con virtualización (VT-x/AMD-V), 16 GB de RAM y unos 100 GB libres en SSD.
Visual Studio Code	Editor de código, con extensiones para PHP, Laravel y Blade.
PHP 8.5, Composer y MySQL 8.4 en local	Instalados con Laravel Herd o Laragon para ejecutar la aplicación mientras se programa, sin depender del servidor.
Node.js y npm	Para compilar los recursos de la interfaz con Vite.
Git y GitHub	Control de versiones y copia remota del código.

Elemento	Uso
Navegador web (Firefox / Chrome)	Pruebas de la interfaz y herramientas de desarrollador.
DBeaver o MySQL Workbench	Consulta y revisión de la base de datos durante el desarrollo.

Entorno de pruebas y despliegue

Se montará en VirtualBox una pequeña red que simula la de una organización real. El esquema completo está en el Anexo I y el direccionamiento en el Anexo II.

Máquinas virtuales del entorno de pruebas

Máquina	Recursos	Función
srv-web	Ubuntu Server 26.04 LTS 2 vCPU · 2 GB RAM · 25 GB	Servidor de producción: Apache, PHP-FPM, Laravel y MySQL. Tiene UFW, Fail2ban, el agente de Zabbix y el script de copias.
srv-backup	Ubuntu Server 26.04 LTS 2 vCPU · 2 GB RAM · 30 GB	Servidor de administración: recibe y guarda las copias de seguridad y aloja el servidor Zabbix con su interfaz web.
cliente-usuario	Lubuntu 1 vCPU · 2 GB RAM	Simula el puesto de un empleado que registra y consulta incidencias desde el navegador.
cliente-técnico	Windows 10/11 2 vCPU · 4 GB RAM	Simula el puesto del técnico: gestiona incidencias y administra los servidores por SSH.

No será necesario hardware de red físico: el switch y el router los proporciona la Red NAT de VirtualBox. Las máquinas cliente no tienen por qué estar encendidas a la vez, lo que reduce el consumo de memoria del equipo anfitrión. En una organización real, srv-web sería un servidor físico pequeño o un servidor virtual (VPS), con la misma configuración.

Configuración de red

- Red NAT de VirtualBox «red_incidencias» con direccionamiento 192.168.50.0/24 y puerta de enlace 192.168.50.1.
- Los servidores tendrán IP fija configurada con Netplan; los clientes recibirán IP por DHCP.
- La aplicación se publicará con el nombre incidencias.local, que los clientes resolverán mediante su archivo hosts.
- Acceso a Internet desde las máquinas solo para instalar paquetes y actualizaciones.

9. Fases de desarrollo

El proyecto seguirá las fases propuestas, adaptadas a sus tareas concretas. Las fechas son una estimación y podrán ajustarse según el calendario del curso.

Planificación por fases

Fase	Tareas principales	Periodo estimado
1. Planificación inicial	Definir el alcance y los objetivos, redactar el anteproyecto, preparar el repositorio Git y organizar el calendario de trabajo.	Noviembre 2026
2. Análisis y diseño	Detallar los requisitos y los permisos de cada rol, cerrar el flujo de estados, completar el modelo Entidad-Relación, refinar las maquetas y diseñar la red y el direccionamiento.	Noviembre – diciembre 2026
3. Desarrollo e implementación	**Infraestructura:** crear las máquinas virtuales, instalar Ubuntu Server, Apache, PHP y MySQL. **Aplicación:** migraciones y seeders, inicio de sesión, perfiles, registro y gestión de incidencias, comentarios, adjuntos, historial y panel del técnico. **Administración:** HTTPS, UFW, SSH y Fail2ban, script de copias con cron y monitorización con Zabbix.	Enero – marzo 2027
4. Pruebas y validación	Probar cada función con los dos roles, comprobar que un usuario no puede acceder a lo que no le corresponde, intentar subir archivos no permitidos, restaurar una copia de seguridad en limpio y forzar alertas en Zabbix.	Abril 2027
5. Despliegue y puesta en marcha	Despliegue final en srv-web desde GitHub, carga de datos de ejemplo, pruebas desde los clientes y redacción de los manuales de usuario y de administración.	Abril – mayo 2027
6. Mantenimiento y soporte	Aplicar actualizaciones, revisar registros y alertas, corregir errores detectados, completar la memoria final y preparar la defensa del proyecto.	Mayo – junio 2027

Desglose de tareas

Cada fase se ha dividido en tareas concretas. Cada tarea tiene un identificador (el mismo que se usa en el tablero de Jira), una descripción, las tareas que requiere, es decir, de las que depende (no puede empezar hasta que estas terminen), las fechas previstas, una estimación en horas y el criterio que indica cuándo se puede dar por terminada. En total se han estimado 204 horas de trabajo.

Fase 1. Planificación inicial (33 h)

ID	Tarea y descripción	Requiere	Inicio / fin	Horas	Criterio de finalización
GI-01	**Definir la idea, el alcance y los objetivos** Elegir el problema a resolver, delimitar qué entra en la primera versión y redactar los objetivos generales y específicos.	—	19/10/26 23/10/26	6 h	Título, descripción, justificación y objetivos redactados en el anteproyecto.

ID	Tarea y descripción	Requiere	Inicio / fin	Horas	Criterio de finalización
GI-02	**Redactar y entregar el anteproyecto (Entregable 2)** Completar los once apartados del anteproyecto y corregirlo según los comentarios del profesor.	GI-01	26/10/26 06/11/26	14 h	Anteproyecto entregado y comentarios del profesor aplicados.
GI-03	**Crear el tablero Kanban en Jira** Crear el proyecto en Jira con las columnas Pendiente, En curso, En revisión o pruebas y Finalizado, cargar todas las tareas y compartirlo con el profesor.	GI-02	09/11/26 10/11/26	3 h	Tablero con todas las tareas y enlace accesible para el profesor.
GI-04	**Crear el repositorio en GitHub** Crear el repositorio público con su estructura de carpetas (docs, diseño, web) y subir la documentación inicial.	GI-02	09/11/26 09/11/26	2 h	Repositorio público con la estructura de carpetas y el anteproyecto subido.
GI-05	**Redactar el README.md** Presentación resumida del proyecto: descripción, justificación, objetivos, tecnologías, funcionalidades, planificación, estado, diseño y enlaces.	GI-04	10/11/26 12/11/26	3 h	README con los diez apartados pedidos y todos los enlaces funcionando.
GI-06	**Publicar la web del proyecto en GitHub Pages** Página HTML/CSS de presentación del proyecto publicada con GitHub Pages.	GI-04	10/11/26 13/11/26	5 h	Web accesible desde la URL de GitHub Pages con los enlaces al repositorio y a Jira.

Fase 2. Análisis y diseño (28 h)

ID	Tarea y descripción	Requiere	Inicio / fin	Horas	Criterio de finalización
GI-07	**Detallar requisitos y permisos por perfil** Listar qué puede hacer el usuario y qué puede hacer el técnico en cada pantalla (matriz de permisos).	GI-02	16/11/26 18/11/26	4 h	Matriz de permisos revisada y coherente con los prototipos.
GI-08	**Cerrar el flujo de estados de una incidencia** Definir las transiciones permitidas entre Abierta, Asignada, En curso, En espera, Resuelta y Cerrada, y quién puede hacer cada cambio.	GI-07	19/11/26 20/11/26	2 h	Diagrama o tabla de transiciones documentado en el repositorio.

ID	Tarea y descripción	Requiere	Inicio / fin	Horas	Criterio de finalización
GI-09	**Prototipos de todas las pantallas** Maquetas de cada opción del menú del usuario y del técnico, con su explicación.	GI-01	02/11/26 13/11/26	10 h	Una maqueta por opción del menú, subida a docs/diseno/prototipos.
GI-10	**Diagrama de arquitectura y especificación técnica** Diagrama con servidores, sistemas, servicios, red, IP, cortafuegos, base de datos y clientes, junto con la tabla de componentes.	GI-02	09/11/26 13/11/26	4 h	Diagrama (editable y PNG) y tabla incluidos en el anteproyecto y en el repositorio.
GI-11	**Modelo Entidad-Relación definitivo y paso a tablas** Revisar el modelo E-R y definir tablas, tipos de datos, claves y valores ENUM.	GI-07, GI-08	23/11/26 27/11/26	5 h	Esquema de tablas listo para crear las migraciones.
GI-12	**Plan de direccionamiento y de seguridad** Cerrar IP, nombres de máquina, puertos abiertos, reglas de cortafuegos y política de copias.	GI-10	30/11/26 04/12/26	3 h	Tabla de IP y reglas de cortafuegos documentadas.

Fase 3. Desarrollo e implementación (84 h)

ID	Tarea y descripción	Requiere	Inicio / fin	Horas	Criterio de finalización
GI-13	**Crear las máquinas virtuales y la Red NAT** Crear en VirtualBox la Red NAT red_incidencias y las máquinas srv-web, srv-backup, cliente-usuario y cliente-técnico.	GI-12	11/01/27 12/01/27	4 h	Las cuatro máquinas se ven entre sí por la red 192.168.50.0/24.
GI-14	**Instalar Ubuntu Server 26.04 y configurar red y SSH** Instalar el sistema en srv-web y srv-backup, IP fija con Netplan, actualizar y configurar SSH con clave sin acceso root.	GI-13	13/01/27 15/01/27	4 h	Acceso por SSH con clave a los dos servidores desde cliente-técnico.
GI-15	**Instalar Apache, PHP 8.5-FPM y MySQL 8.4** Instalar la pila web en srv-web, crear la base de datos y un usuario con permisos solo sobre ella.	GI-14	18/01/27 22/01/27	5 h	Página de prueba PHP servida por Apache y conexión a MySQL con el usuario de la aplicación.

ID	Tarea y descripción	Requiere	Inicio / fin	Horas	Criterio de finalización
GI-16	**Crear el proyecto Laravel 13** Crear el proyecto, configurar .env, Bootstrap y AdminLTE y subirlo al repositorio.	GI-04	18/01/27 22/01/27	3 h	Proyecto arrancando en local y subido a GitHub.
GI-17	**Migraciones y datos iniciales** Crear las tablas USUARIOS, INCIDENCIAS, COMENTARIOS e HISTORIAL_ESTADOS con sus claves, y seeders con usuarios de prueba.	GI-11, GI-16	25/01/27 29/01/27	5 h	php artisan migrate --seed crea la base de datos completa sin errores.
GI-18	**Inicio de sesión y control de acceso por perfil** Login, cierre de sesión con confirmación y protección de rutas según el perfil (usuario o técnico).	GI-17	01/02/27 05/02/27	6 h	Cada perfil solo accede a sus pantallas; un usuario no puede abrir rutas del técnico.
GI-19	**Pantallas del usuario: Mis incidencias y Nueva incidencia** Formulario de registro con validación y listado de las incidencias propias con su estado.	GI-18	08/02/27 12/02/27	8 h	Un usuario crea una incidencia y la ve en su listado con estado Abierta.
GI-20	**Comentarios con archivo adjunto** Añadir comentarios (internos o visibles) con un adjunto opcional guardado en la zona privada de Laravel.	GI-19	15/02/27 19/02/27	6 h	Se suben imágenes y PDF de hasta 5 MB; otros tipos se rechazan.
GI-21	**Pantallas del técnico: Panel, Todas, Asignadas y Detalle** Panel con indicadores, listados con filtros, detalle de la incidencia, asignación de técnico, cambio de estado e historial.	GI-19	22/02/27 05/03/27	14 h	Cada cambio de estado queda guardado en HISTORIAL_ESTADOS con fecha y autor.
GI-22	**Gestión de usuarios y Mi perfil** Alta, edición y desactivación de cuentas (técnico) y edición de datos y contraseña (usuario).	GI-18	08/03/27 12/03/27	6 h	Una cuenta desactivada no puede iniciar sesión.
GI-23	**Módulo de informes** Informe por periodo, categoría y técnico con totales, tiempo medio de resolución y gráficas.	GI-21	15/03/27 19/03/27	6 h	Los datos del informe coinciden con una consulta SQL hecha a mano.

ID	Tarea y descripción	Requiere	Inicio / fin	Horas	Criterio de finalización
GI-24	**HTTPS con CA propia y VirtualHost** Crear la CA con OpenSSL, emitir el certificado de incidencias.local, configurar el VirtualHost 443 y redirigir el 80.	GI-15	01/03/27 05/03/27	4 h	https://incidencias.local abre sin aviso en los clientes y http redirige a https.
GI-25	**Cortafuegos UFW y Fail2ban** Política de denegar por defecto, abrir solo 80/443, SSH desde la red interna y 10050 desde srv-backup; Fail2ban para SSH.	GI-15	08/03/27 12/03/27	3 h	ufw status muestra solo las reglas previstas y Fail2ban bloquea tras varios intentos fallidos.
GI-26	**Script de copias de seguridad con cron y rsync** Script Bash con mysqldump, copia de adjuntos y .env, envío a srv-backup y rotación de 7 días.	GI-15, GI-17	15/03/27 19/03/27	5 h	La copia aparece cada día en srv-backup y se borran las de más de 7 días.
GI-27	**Monitorización con Zabbix** Instalar Zabbix Server en srv-backup y el agente en srv-web; plantillas de Linux, Apache y MySQL y alertas.	GI-14	22/03/27 26/03/27	5 h	Zabbix muestra las gráficas de srv-web y genera una alerta al parar Apache.

Fase 4. Pruebas y validación (19 h)

ID	Tarea y descripción	Requiere	Inicio / fin	Horas	Criterio de finalización
GI-28	**Pruebas funcionales por perfil** Recorrer todas las pantallas con un usuario y con un técnico siguiendo una lista de casos de prueba.	GI-20, GI-21, GI-22, GI-23	05/04/27 14/04/27	10 h	Todos los casos de prueba superados o con su error registrado en Jira.
GI-29	**Pruebas de seguridad** Acceso a rutas sin permiso, subida de archivos no permitidos, escaneo de puertos con nmap y prueba de Fail2ban.	GI-24, GI-25, GI-28	15/04/27 16/04/27	4 h	Ninguna prueba consigue acceder a datos o puertos no permitidos.
GI-30	**Prueba de restauración de una copia** Restaurar la base de datos y los adjuntos en limpio a partir de una copia de srv-backup.	GI-26	19/04/27 21/04/27	3 h	La aplicación funciona con los datos restaurados.
GI-31	**Prueba de alertas de Zabbix** Provocar fallos controlados (parar Apache o MySQL, llenar disco) y comprobar las alertas.	GI-27	19/04/27 23/04/27	2 h	Cada fallo provocado genera su alerta en Zabbix.

Fase 5. Despliegue y puesta en marcha (15 h)

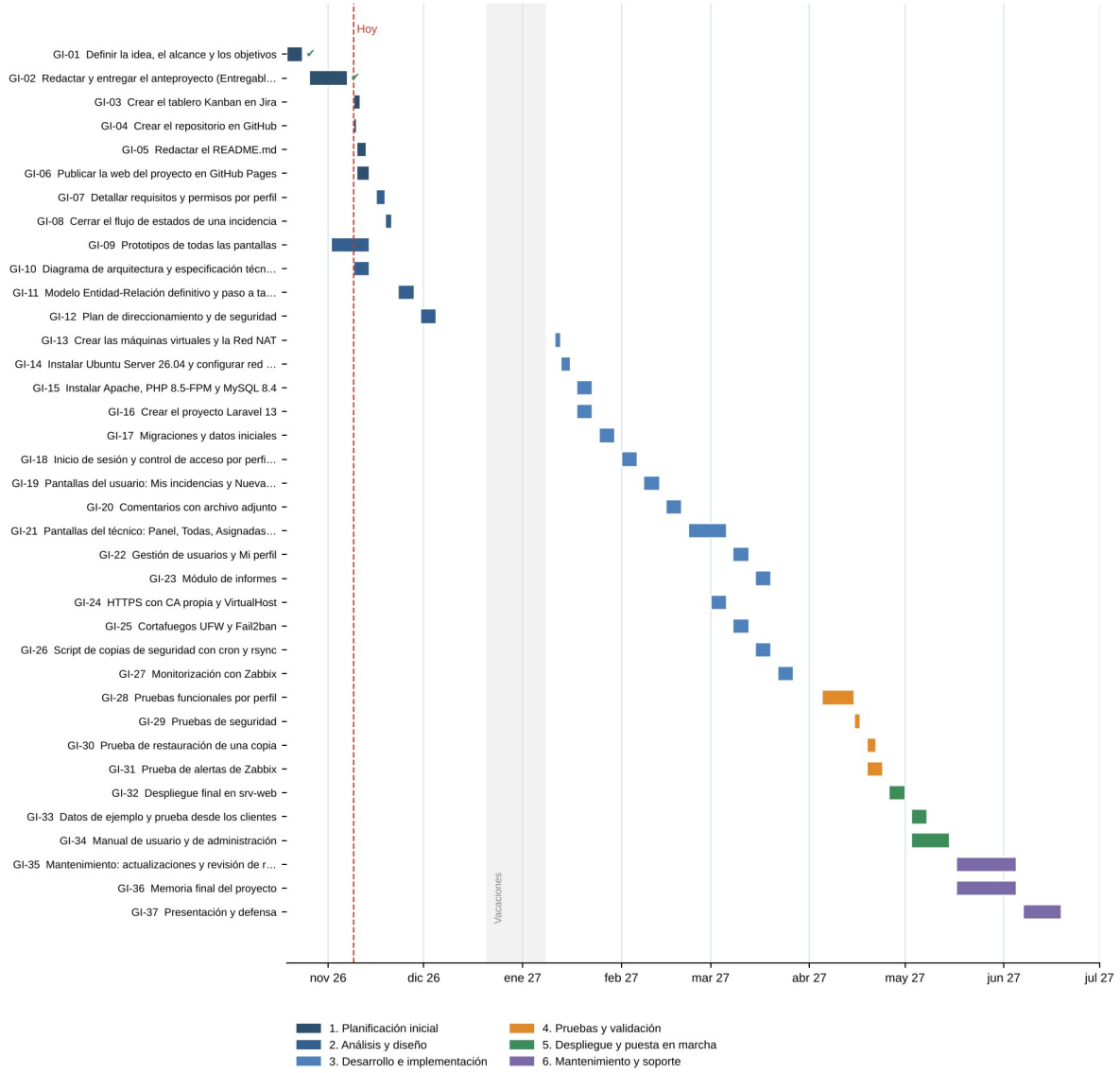
ID	Tarea y descripción	Requiere	Inicio / fin	Horas	Criterio de finalización
GI-32	**Despliegue final en srv-web** Clonar la versión final desde GitHub, composer install --no-dev, compilar recursos y migrar.	GI-28, GI-29	26/04/27 30/04/27	4 h	La aplicación funciona en https://incidencias.local con la última versión del repositorio.
GI-33	**Datos de ejemplo y prueba desde los clientes** Cargar usuarios e incidencias de ejemplo y probar desde cliente-usuario y cliente-técnico.	GI-32	03/05/27 07/05/27	3 h	Ambos clientes usan la aplicación sin errores.
GI-34	**Manual de usuario y de administración** Guías con capturas para el usuario, el técnico y el administrador del servidor.	GI-32	03/05/27 14/05/27	8 h	Manuales subidos al repositorio en docs/manuales.

Fase 6. Mantenimiento y soporte (25 h)

ID	Tarea y descripción	Requiere	Inicio / fin	Horas	Criterio de finalización
GI-35	**Mantenimiento: actualizaciones y revisión de registros** Aplicar actualizaciones de seguridad, revisar registros de Apache y alertas de Zabbix y corregir errores.	GI-32	17/05/27 04/06/27	4 h	Sistema actualizado y sin errores pendientes en Jira.
GI-36	**Memoria final del proyecto** Redactar la memoria a partir del anteproyecto, con el desarrollo, las pruebas y las conclusiones.	GI-34	17/05/27 04/06/27	15 h	Memoria entregada en Moodle.
GI-37	**Presentación y defensa** Preparar la presentación y la demostración en directo de la aplicación y la infraestructura.	GI-36	07/06/27 18/06/27	6 h	Presentación preparada y demo ensayada.

Cronograma

El siguiente diagrama de Gantt muestra el orden de las tareas en el tiempo. Las tareas siguen el orden de sus dependencias: por ejemplo, no se programa la aplicación hasta tener el modelo de datos cerrado, y no se hacen las pruebas de seguridad hasta tener configurados HTTPS y el cortafuegos. Durante las vacaciones de Navidad no se planifica trabajo.



Cronograma de las tareas del proyecto (diagrama de Gantt)

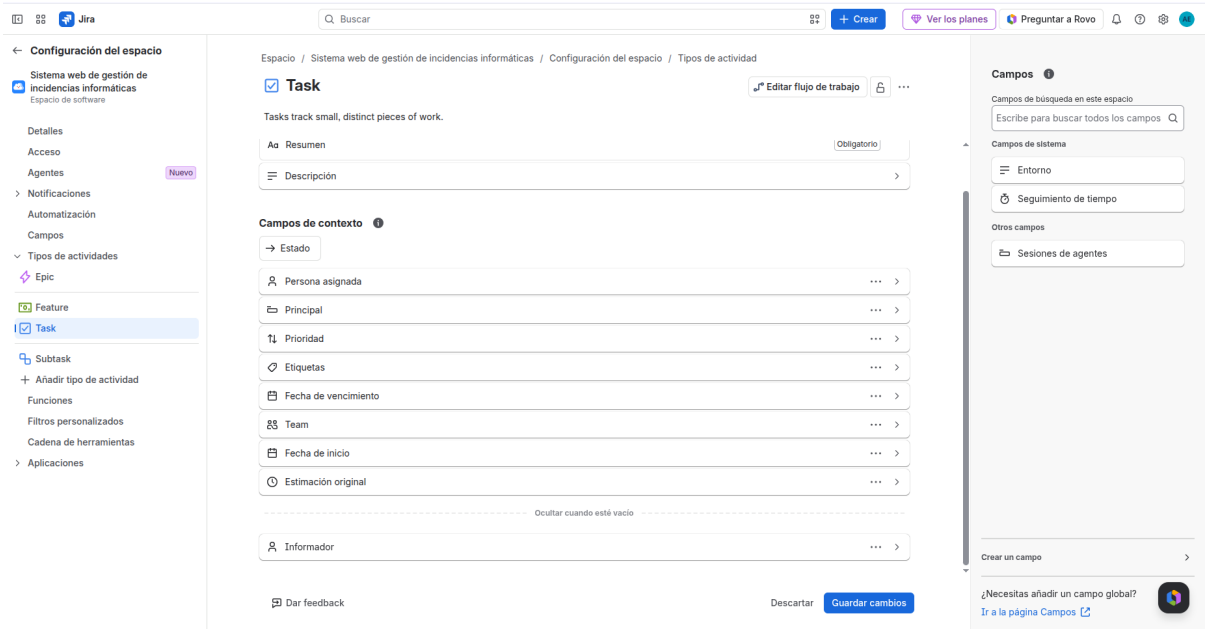
Organización y seguimiento del trabajo

Para organizar el trabajo y llevar el seguimiento se usan tres herramientas relacionadas entre sí: un tablero Kanban en Jira con todas las tareas, un repositorio en GitHub con toda la documentación y una página web publicada con GitHub Pages. Los enlaces están en el Anexo V.

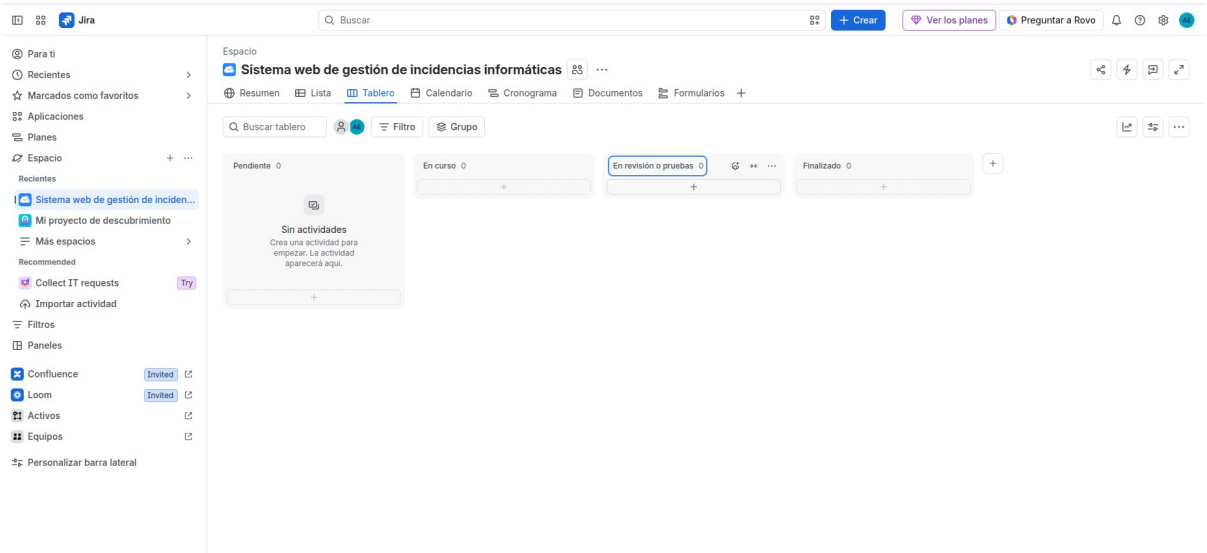
Tablero Kanban en Jira

En Jira se ha creado el espacio «Sistema web de gestión de incidencias informáticas» con un tablero Kanban. El tablero traía al principio las columnas Idea, Por hacer, En curso, En revisión y Finalizado, y se han cambiado para dejar las cuatro que se van a usar: Pendiente (tareas sin empezar), En curso (lo que se está haciendo), En revisión o pruebas (tareas hechas que falta comprobar) y Finalizado (tareas terminadas y comprobadas).

También se ha añadido a las tareas el campo «Estimación original», que no venía por defecto, para poder apuntar las horas de cada una.

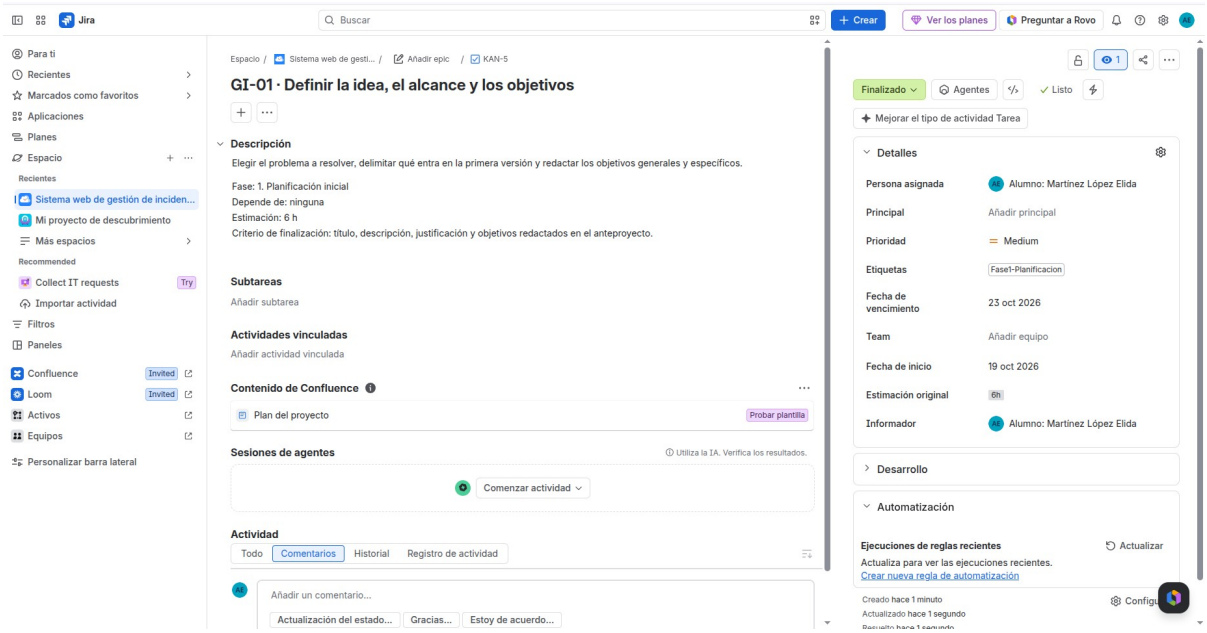


Campos de la tarea con «Estimación original» añadida



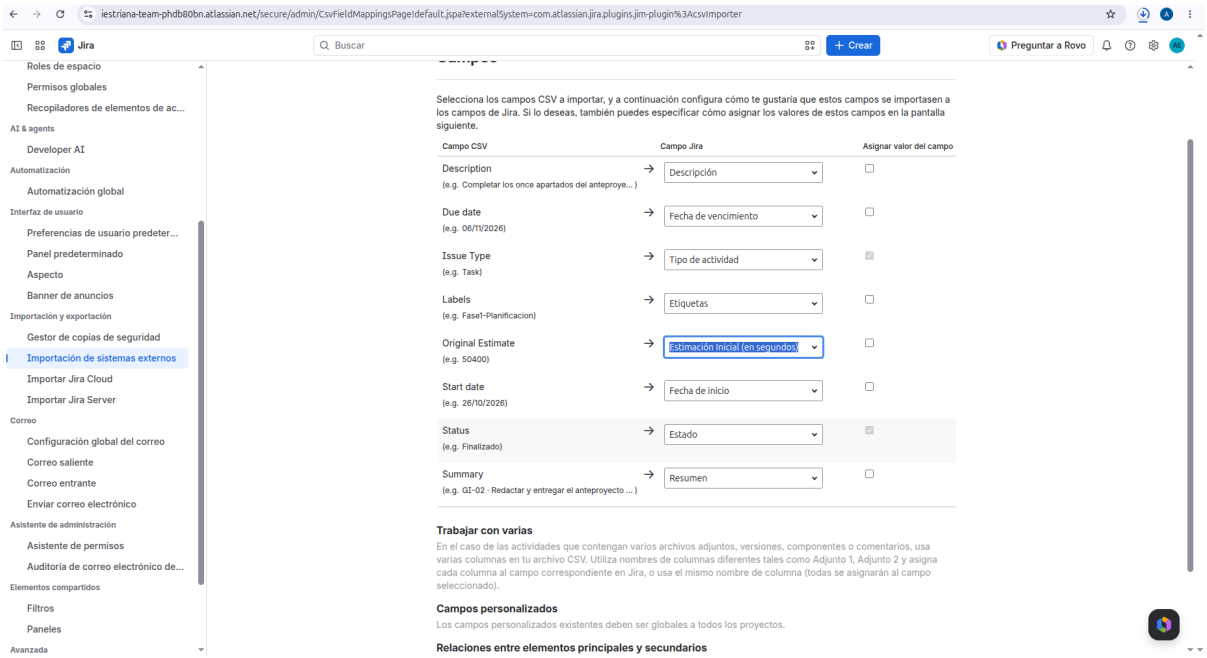
Tablero de Jira con las cuatro columnas

En el tablero están las 37 tareas de la tabla anterior. Cada una lleva su identificador y nombre en el resumen (por ejemplo «GI-01 · Definir la idea, el alcance y los objetivos»), la descripción, la fase como etiqueta, las fechas de inicio y fin, la estimación en horas y, en la descripción, de qué tareas depende y su criterio de finalización. La primera tarea se creó a mano para comprobar que estaban todos los campos:

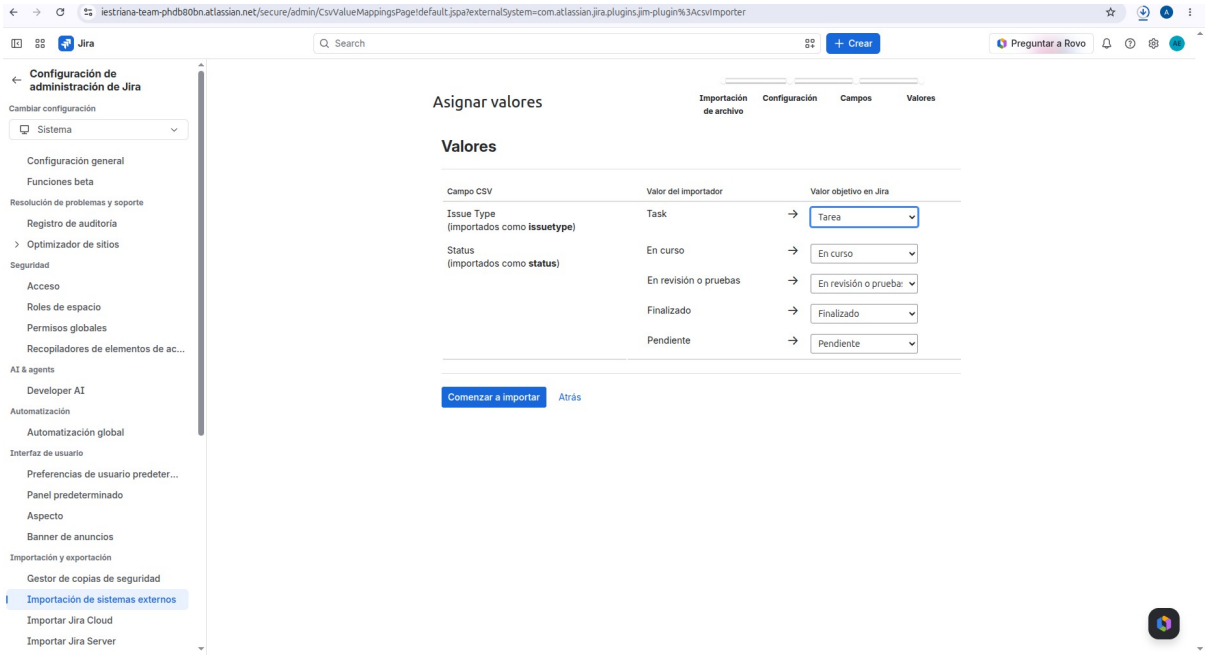


Tarea GI-01 abierta, con su descripción y sus detalles

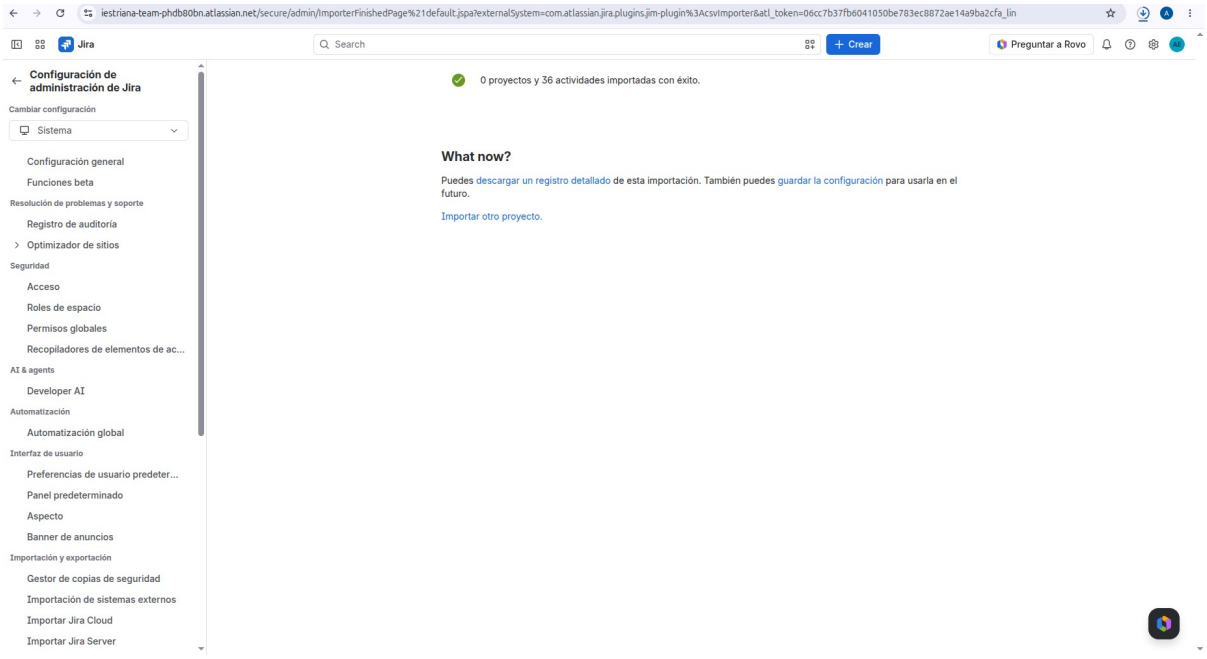
Las otras 36 se cargaron de una vez con el importador CSV de Jira, a partir de un archivo con toda la planificación. En la importación se indicó qué columna del archivo iba en cada campo, a qué columna del tablero correspondía cada estado y el formato de las fechas (dd/MM/yyyy). La opción nueva de importación creaba un espacio nuevo, así que se usó la importación anterior, que permite cargar las tareas en un espacio que ya existe.



Asignación de las columnas del archivo a los campos de Jira

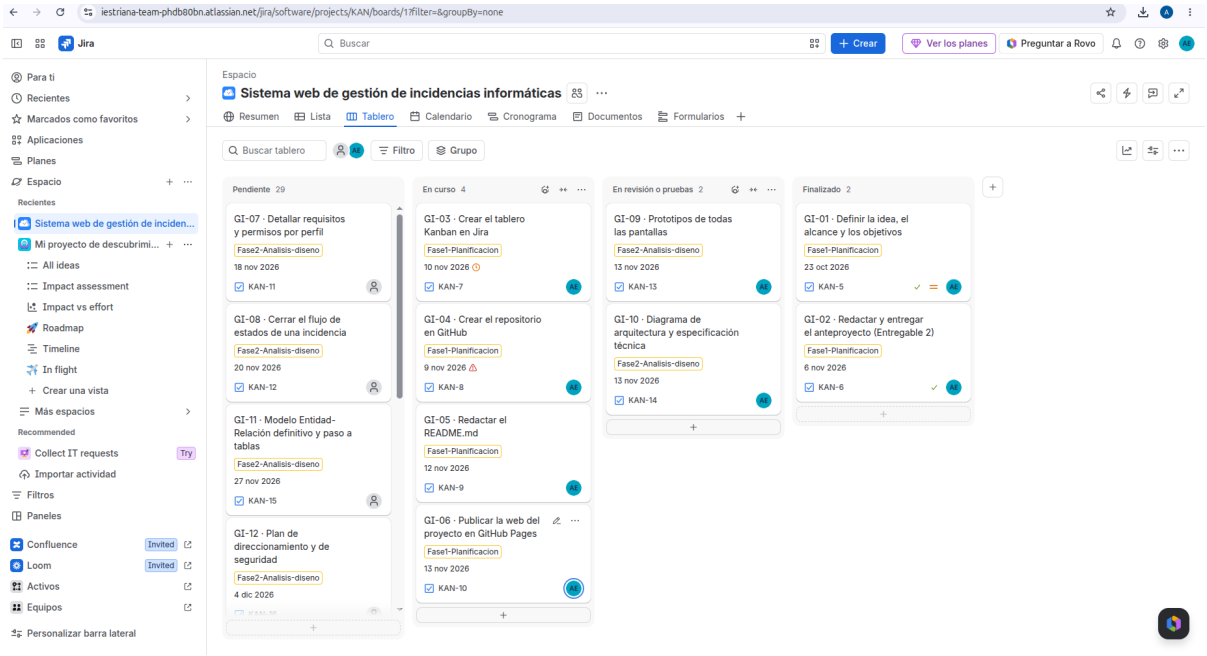


Asignación de los estados del archivo a las columnas del tablero



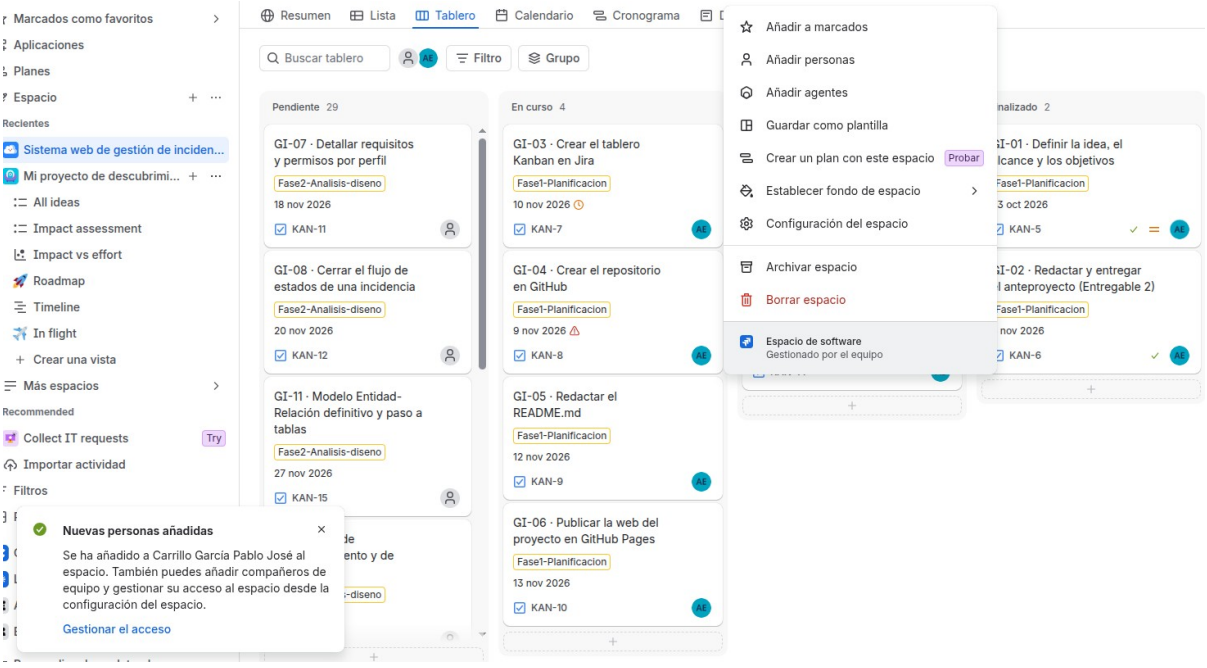
Importación terminada: 36 actividades importadas con éxito

El tablero muestra el estado real del proyecto: las dos primeras tareas están finalizadas, las de organización (Jira, repositorio, README y web) están en curso, los prototipos y el diagrama de arquitectura están en revisión y el resto está pendiente. Las tareas en las que se está trabajando están asignadas a la alumna, y las tarjetas se irán moviendo de columna a medida que avance el proyecto.



Tablero de Jira con todas las tareas en su columna

El tablero se ha compartido con el profesor, Pablo José Carrillo García, con el rol de Lector, de forma que puede ver las tareas y su estado y comentar, pero no modificar nada.



Aviso de Jira al añadir al profesor al espacio

Repositorio en GitHub

El repositorio público gestor-incidencias reúne todo el proyecto. En la raíz están el README.md, que es la presentación del repositorio, y el index.html de la página web. Dentro de la carpeta docs está la documentación: el anteproyecto, la planificación (tabla de tareas, cronograma y el CSV importado en Jira) y los diseños (prototipos y diagramas, cada uno con su archivo editable). Más adelante también contendrá el código de la aplicación.

```
gestor-incidencias/
├── README.md                    presentación del proyecto
├── index.html                   página web (GitHub Pages)
└── docs/
    ├── Anteproyecto.pdf
    ├── planificacion/           tareas, cronograma y CSV de Jira
    └── diseno/
        ├── diagramas/           arquitectura, modelo E-R y red (+ editables)
        └── prototipos/          11 pantallas (+ fuentes HTML/CSS)
```

El README resume el proyecto sin copiar el anteproyecto, usando títulos, listas, tablas y enlaces de Markdown. Incluye el nombre del proyecto, la descripción, la justificación, los objetivos, las tecnologías, las funcionalidades, la planificación, el estado actual, los enlaces al diseño técnico (diagramas y prototipos) y los enlaces al tablero de Jira y a la web.

[Pega aquí la captura: página principal del repositorio con el README]

Página web en GitHub Pages

La página web está hecha con HTML y CSS, se adapta al móvil y se publica directamente desde el repositorio con GitHub Pages. Sirve como presentación inicial del proyecto e incluye el nombre y título del proyecto, el problema que resuelve, los objetivos, las funcionalidades previstas, las tecnologías, el estado actual, el diagrama de arquitectura, algunas pantallas del prototipo, el cronograma y los enlaces al repositorio y al tablero de Jira.

[Pega aquí la captura: página web publicada en GitHub Pages]

Estado actual del proyecto

El proyecto se encuentra en la **fase 1 (Planificación inicial)**, terminando la organización del trabajo, y ha empezado la fase 2 con los prototipos y el diagrama de arquitectura. El anteproyecto está entregado y corregido, y se están preparando el tablero de Jira, el repositorio de GitHub y la página web.

Posibles mejoras futuras

Quedan fuera del alcance de la primera versión, pero podrían añadirse si sobra tiempo o en una versión posterior:

- Avisos por correo electrónico cuando se crea, asigna o resuelve una incidencia.
- Categorías y prioridades configurables desde el panel de administración.
- Exportar los informes a PDF o a hoja de cálculo.
- Inicio de sesión integrado con el directorio de la organización (LDAP o Active Directory).

10. Módulos del ciclo formativo relacionados

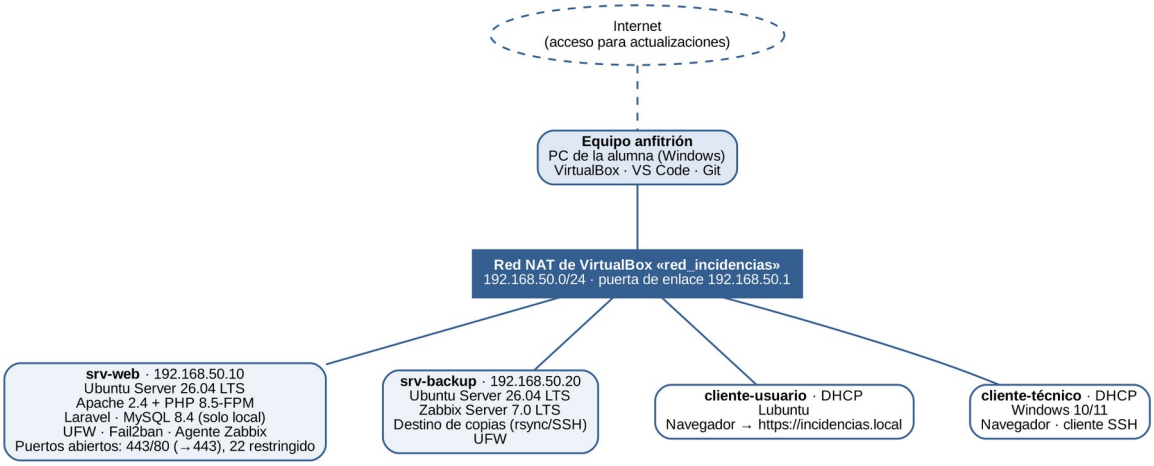
El proyecto está directamente relacionado con los siguientes módulos del ciclo:

Relación del proyecto con los módulos del ciclo

Módulo	Curso	Relación con el proyecto
Implantación de Sistemas Operativos	1º	Instalación y configuración de Ubuntu Server, Lubuntu y Windows en las máquinas virtuales; gestión de usuarios, permisos y servicios.
Planificación y Administración de Redes	1º	Diseño de la red del entorno de pruebas, direccionamiento IP, puerta de enlace y configuración de red de servidores y clientes.
Fundamentos de Hardware	1º	Dimensionado de los recursos de las máquinas virtuales y del equipo anfitrión; además, gran parte de las incidencias que gestionará el sistema son de hardware y periféricos.
Gestión de Bases de Datos	1º	Diseño del modelo Entidad-Relación, paso a tablas, claves primarias y foráneas y consultas SQL.
Lenguajes de Marcas y Sistemas de Gestión de Información	1º	Estructura de las páginas con HTML y diseño con CSS.
Administración de Sistemas Operativos	2º	Scripts en Bash, tareas programadas con cron, gestión de servicios y revisión de registros del sistema.
Servicios de Red e Internet	2º	Servidor web Apache, VirtualHosts, HTTPS con certificados, SSH y resolución de nombres.
Implantación de Aplicaciones Web	2º	Desarrollo de la aplicación en PHP con Laravel y su despliegue en el servidor web.
Administración de Sistemas Gestores de Bases de Datos	2º	Instalación y configuración de MySQL, usuarios con privilegios mínimos y copias de seguridad con mysqldump.
Seguridad y Alta Disponibilidad	2º	Cortafuegos UFW, Fail2ban, cifrado HTTPS, copias de seguridad y su restauración, y monitorización con Zabbix.
Itinerario Personal para la Empleabilidad I y II	1º y 2º	Planificación del proyecto, análisis de las necesidades de una pequeña organización y estudio de las soluciones existentes en el mercado.

11. Anexos y bibliografía

Anexo I. Esquema de red del entorno de pruebas



Esquema de red del entorno de pruebas en VirtualBox

Anexo II. Plan de direccionamiento y servicios

Direccionamiento y servicios de cada máquina

Máquina	Dirección IP	Sistema operativo	Servicios y puertos
Puerta de enlace	192.168.50.1	Red NAT VirtualBox	Router/NAT y DHCP para los clientes.
srv-web	192.168.50.10 (fija)	Ubuntu Server 26.04 LTS	Apache 80 → redirige a 443 (HTTPS) · SSH 22 solo desde la red interna · MySQL 3306 solo local · agente Zabbix 10050 solo desde srv-backup.
srv-backup	192.168.50.20 (fija)	Ubuntu Server 26.04 LTS	SSH 22 (recepción de copias con rsync) · Zabbix Server 10051 · interfaz web de Zabbix 443.
cliente-usuario	DHCP (192.168.50.100–150)	Lubuntu	Navegador web.
cliente-técnico	DHCP (192.168.50.100–150)	Windows 10/11	Navegador web y cliente SSH.

Anexo III. Matriz de permisos por rol

Qué puede hacer cada rol en la aplicación

Acción	Usuario	Técnico/Admin.
Registrar una incidencia y adjuntar archivos	Sí	Sí
Ver sus propias incidencias y su estado	Sí	Sí
Ver todas las incidencias de la organización	No	Sí
Añadir comentarios a una incidencia	Sí (solo las suyas)	Sí
Ver comentarios internos	No	Sí

Acción	Usuario	Técnico/Admin.
Asignar técnico, cambiar prioridad y estado	No	Sí
Registrar la solución aplicada	No	Sí
Confirmar el cierre o reabrir una incidencia	Sí (solo las suyas)	Sí
Gestionar usuarios (alta, edición y desactivación de cuentas)	No	Sí
Consultar informes	No	Sí

Anexo IV. Esquema del script de copias de seguridad

Primera versión orientativa del script que se ejecutará cada noche con cron en srv-web. Se completará y probará durante la fase de desarrollo:

```
#!/bin/bash
# backup_incidencias.sh - copia diaria de la BD y de los adjuntos
FECHA=$(date +%F)
ORIGEN_APP=/var/www/incidencias
DESTINO=/var/backups/incidencias/$FECHA
mkdir -p "$DESTINO"

# 1. Base de datos (credenciales en ~/.my.cnf, no en el script)
mysqldump --single-transaction incidencias | gzip > "$DESTINO/bd.sql.gz"

# 2. Archivos adjuntos y configuración
tar -czf "$DESTINO/adjuntos.tar.gz" -C "$ORIGEN_APP" storage/app
cp "$ORIGEN_APP/.env" "$DESTINO/env.bak"

# 3. Envío al servidor de copias y rotación (se conservan 7 días)
rsync -az /var/backups/incidencias/ backup@192.168.50.20:/srv/copias/incidencias/
find /var/backups/incidencias -mindepth 1 -maxdepth 1 -mtime +7 -exec rm -rf {} +

# Tarea en cron (todos los días a las 02:30):
# 30 2 * * * /usr/local/bin/backup_incidencias.sh >> /var/log/backup_incidencias.log
2>&1
```

Anexo V. Enlaces del proyecto

Recurso	Enlace
Repositorio de GitHub	https://github.com/elimartinezlopez/gestor-incidencias
Página web (GitHub Pages)	https://elimartinezlopez.github.io/gestor-incidencias/
Tablero Kanban de Jira	https://iestriana-team-phdb80bn.atlassian.net/jira/software/projects/KAN/boards/1

Bibliografía

Fuentes consultadas para la elaboración del anteproyecto (consultadas en noviembre de 2026):

[1] Laravel. (s. f.). Laravel Documentation. https://laravel.com/docs

[2] The PHP Group. (s. f.). Manual de PHP. https://www.php.net/manual/es/

[3] Oracle. (s. f.). MySQL 8.4 Reference Manual. https://dev.mysql.com/doc/refman/8.4/en/

[4] Bootstrap. (s. f.). Bootstrap 5.3 Documentation. https://getbootstrap.com/docs/5.3/

[5] AdminLTE. (s. f.). AdminLTE – Bootstrap Admin Dashboard Template. https://adminlte.io/

[6] The Apache Software Foundation. (s. f.). Apache HTTP Server Version 2.4 Documentation. https://httpd.apache.org/docs/2.4/

[7] Canonical. (s. f.). Ubuntu Server documentation. https://documentation.ubuntu.com/server/

[8] Oracle. (s. f.). Oracle VirtualBox User Guide. https://www.virtualbox.org/manual/

[9] Zabbix. (s. f.). Zabbix 7.0 Documentation. https://www.zabbix.com/documentation/7.0/

[10] Fail2ban. (s. f.). Repositorio oficial de Fail2ban. https://github.com/fail2ban/fail2ban

[11] Git. (s. f.). Git Documentation. https://git-scm.com/doc

[12] OWASP Foundation. (s. f.). OWASP Top Ten. https://owasp.org/www-project-top-ten/

[13] osTicket. (s. f.). osTicket – Support Ticket System. https://osticket.com/

[14] GLPI Project. (s. f.). GLPI – Gestionnaire Libre de Parc Informatique. https://glpi-project.org/

[15] Zammad. (s. f.). Zammad – Helpdesk and Ticket System. https://zammad.com/

[16] Atlassian. (s. f.). Jira Service Management. https://www.atlassian.com/software/jira/service-management

[17] Atlassian. (s. f.). Documentación de Jira: tableros Kanban. https://support.atlassian.com/jira-software-cloud/

[18] GitHub. (s. f.). Documentación de GitHub Pages. https://docs.github.com/es/pages